



Búsqueda con retroceso (*backtracking*)

Simona Bernardi

Universidad de Zaragoza

Este documento está sujeto a una licencia de uso Creative Commons. No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.





Resumen

- Método general
- Ejemplos
 - Suma de subconjuntos
 - El problema de la mochila 0-1
- Técnica *meet-in-the-middle*

Introducción



Búsqueda de una solución o todas las soluciones
que **satisfacen a ciertas condiciones**



Introducción

Búsqueda de una solución o todas las soluciones
que **satisfacen a ciertas condiciones**

- A menudo, el espacio de búsqueda consta de muchas decisiones, cada una tiene varias opciones disponibles
- Una solución es el resultado de una secuencia de decisiones



Introducción

Búsqueda de una solución o todas las soluciones
que **satisfacen a ciertas condiciones**

- A menudo, el espacio de búsqueda consta de muchas decisiones, cada una tiene varias opciones disponibles
- Una solución es el resultado de una secuencia de decisiones

Aplicaciones:

- Analizadores sintácticos (parser)
- Juegos: anagramas, palabras cruzadas, sopas de letras, ajedrez (las 8 reinas), sudoku, atravesar un laberinto ...
- Problemas de planificación, asignación, ...



Planteamiento del problema

Dados:

- 1 Un espacio de búsqueda: $S = \{(x_1, \dots, x_n) \mid x_i \in C_i, i = 1, \dots, n\}$
- 2 Un predicado $P : S \longrightarrow \{true, false\}$

⇒ Buscar una solución (o todas) que satisfaga al predicado:

$$(x_1, \dots, x_n) \mid P(x_1, \dots, x_n) = true$$



Planteamiento del problema

Dados:

- 1 Un espacio de búsqueda: $S = \{(x_1, \dots, x_n) \mid x_i \in C_i, i = 1, \dots, n\}$
- 2 Un predicado $P : S \longrightarrow \{true, false\}$

⇒ Buscar una solución (o todas) que satisfaga al predicado:

$$(x_1, \dots, x_n) \mid P(x_1, \dots, x_n) = true$$

¿Qué tamaño tiene el espacio de búsqueda?



Tipos de restricciones de las tuplas solución

- **Explícitas:** describen el conjunto C_i de valores que puede tomar x_i
 - Definen el espacio de búsqueda: $S = C_1 \times \cdots \times C_n$
- **Implícitas:** describen las relaciones que deben cumplirse entre los x_i
 - Identifican las tuplas que satisfacen el predicado P



Técnicas de búsqueda

Fuerza bruta:

- 1 Generar todas las tuplas $(x_1, \dots, x_n)$ del espacio y
- 2 Seleccionar una tupla (o todas) que satisfaga el predicado P



Técnicas de búsqueda

Fuerza bruta:

- 1 Generar todas las tuplas $(x_1, \dots, x_n)$ del espacio y
- 2 Seleccionar una tupla (o todas) que satisfaga el predicado P

Retroceso (*backtracking*, en inglés):

- Formar cada tupla de manera progresiva, elemento a elemento, comprobando para cada elemento x_i añadido a la tupla que $(x_1, \dots, x_i)$ puede conducir a una tupla completa satisfactoria



Técnicas de búsqueda

Fuerza bruta:

- 1 Generar todas las tuplas $(x_1, \dots, x_n)$ del espacio y
- 2 Seleccionar una tupla (o todas) que satisfaga el predicado P

Retroceso (*backtracking*, en inglés):

- Formar cada tupla de manera progresiva, elemento a elemento, comprobando para cada elemento x_i añadido a la tupla que $(x_1, \dots, x_i)$ puede conducir a una tupla completa satisfactoria
- ☺ Si la tupla parcial $(x_1, \dots, x_i)$ **no** puede conducir a una tupla completa satisfactoria, se evita formar las $|C_{i+1}| \times \dots \times |C_n|$ tuplas que comienzan por $(x_1, \dots, x_i)$



Búsqueda con retroceso

Deben existir unos **predicados acotadores**:

$$P_i : C_1 \times \cdots \times C_i \longrightarrow \{true, false\}, i = 1, \dots, n-1$$

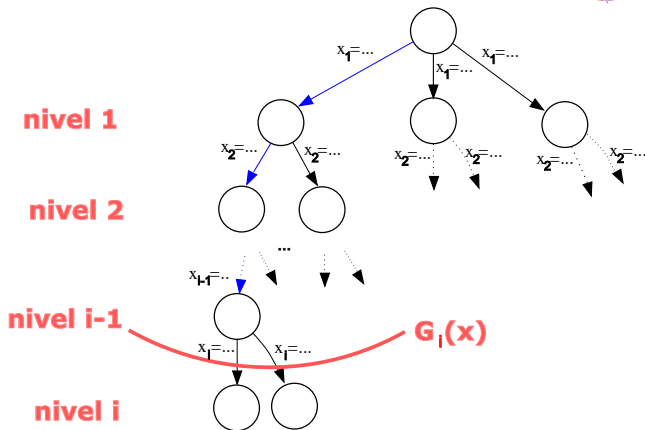
que permiten verificar si la tupla parcial $(x_1, \dots, x_i)$ puede conducir a una solución



Representación del espacio de búsqueda

Caminos y ramificación

- $(x_1, \dots, x_{i-1})$: camino de la raíz hasta un nodo de nivel $i-1$
- $G_i(x)$: conjunto de valores de x_i tales que $(x_1, \dots, x_i)$ es un camino hasta un nodo.





Representación del espacio de búsqueda

Predicados acotadores y soluciones

- Si $P_i(x)$ es falso
 $\Rightarrow (x_1, \dots, x_i)$ no
 puede extenderse
 para alcanzar una
 solución

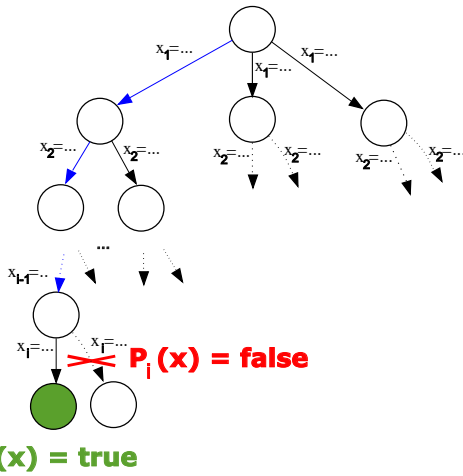
nivel 1

nivel 2

- $R_i(x)$ predicado
 que determina si
 $(x_1, \dots, x_i)$ es
 una solución

nivel i-1

nivel i





Esquema algorítmico

Resumen notación

$(x_1, \dots, x_i)$	un camino de la raíz hasta un nodo del árbol
$G_i(x)$	el conjunto de valores de x_i tales que $(x_1, \dots, x_i)$ es un camino hasta un nodo del árbol.
$P_i(x)$	predicado acotador, si es falso $\Rightarrow$ el camino $(x_1, \dots, x_i)$ no puede conducir a una solución
$R_i(x)$	predicado que determina si un camino $(x_1, \dots, x_i)$ es una solución

- Los candidatos x_i son los valores de $G_i(x)$ tales que $P_i(x) = \text{verdadero}$



Esquema algorítmico

Escribe todas las soluciones. Llamada inicial: `backtracking(x,1);`

```

1  // Pre: x es una tupla de n elementos,  $1 \leq i \leq n$ 
2  //      la tupla  $x[1, \dots, i-1]$  es prometedora
3  // Post: escribe todas las tuplas solución
4
5  void backtracking(tupla x, int i) {
6      for each nodo  $\in G_i(x)$  { //  $G_i(x)$  conjunto de valores de x;
7          if  $P_i(x)$  { // predicado acotador
8              x[i] = nodo;
9              if  $R_i(x)$  { // predicado que determina si es solución
10                 escribir_solucion(x,i);
11             }
12             if  $i < n$  { backtracking(x,i+1); }
13         }
14     }
15 }
```

¿Qué recorrido hace del árbol?

Variantes a la solución propuesta



- Limitar el número de soluciones a una sola



Variantes a la solución propuesta

- Limitar el número de soluciones a una sola
- Forzar a que sólo los nodos hoja puedan significar una solución



Variantes a la solución propuesta

- Limitar el número de soluciones a una sola
- Forzar a que sólo los nodos hoja puedan significar una solución
- Resolver problemas de optimización
 - Hay que guardar la mejor solución encontrada hasta el momento
 - Se mejora la eficiencia de la búsqueda si los predicados acotadores permiten eliminar los nodos de los que se sabe que no pueden llevar a una solución mejor que la ahora disponible.



Eficiencia del algoritmo de búsqueda

Factores

- 1 El tiempo necesario para generar un nodo
- 2 El número de valores en $G_i(x)$
- 3 El tiempo necesario para evaluar los predicados $P_i(x)$
- 4 El número de valores que satisfacen los predicados $P_i(x)$



Eficiencia del algoritmo de búsqueda

Factores

- ❶ El tiempo necesario para generar un nodo
 - ❷ El número de valores en $G_i(x)$
 - ❸ El tiempo necesario para evaluar los predicados $P_i(x)$
 - ❹ El número de valores que satisfacen los predicados $P_i(x)$
- **Buenos predicados** reducen mucho el número de nodos generados
 - A menudo buenos predicados precisan mucho tiempo de evaluación
 - Si se busca una solución y los predicados reducen a un solo nodo generado (solución **voraz**): $O(n)$ nodos a generar en total
 - Caso peor: $O(p(n) \times 2^n)$ o $O(p(n)n!)$, con $p(n)$ polinomio



Suma de subconjuntos

Problema: Dados n números positivos $a_i, 1 \leq i \leq n$, y un número positivo S encontrar todos los subconjuntos de los a_i , cuya suma es S .



Suma de subconjuntos

Problema: Dados n números positivos $a_i, 1 \leq i \leq n$, y un número positivo S encontrar todos los subconjuntos de los a_i , cuya suma es S .

- Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$
- Subconjuntos solución: $\{11, 13, 7\}$, $\{24, 7\}$



Suma de subconjuntos

Representación de las soluciones y restricciones

Problema: Dados n números positivos $a_i, 1 \leq i \leq n$, y un número positivo S encontrar todos los subconjuntos de los a_i , cuya suma es S .



Suma de subconjuntos

Representación de las soluciones y restricciones

Problema: Dados n números positivos $a_i, 1 \leq i \leq n$, y un número positivo S encontrar todos los subconjuntos de los a_i , cuya suma es S .

- Una solución es un vector $(x_1, \dots, x_i), 1 \leq i \leq n$, donde cada $x_j, 1 \leq j \leq i$ es un índice que indica un elemento del conjunto



Suma de subconjuntos

Representación de las soluciones y restricciones

Problema: Dados n números positivos $a_i, 1 \leq i \leq n$, y un número positivo S encontrar todos los subconjuntos de los a_i , cuya suma es S .

- Una solución es un vector $(x_1, \dots, x_i), 1 \leq i \leq n$, donde cada $x_j, 1 \leq j \leq i$ es un índice que indica un elemento del conjunto
- **Restricciones explícitas:** $x_j \in \{1, \dots, n\}, 1 \leq j \leq i$



Suma de subconjuntos

Representación de las soluciones y restricciones

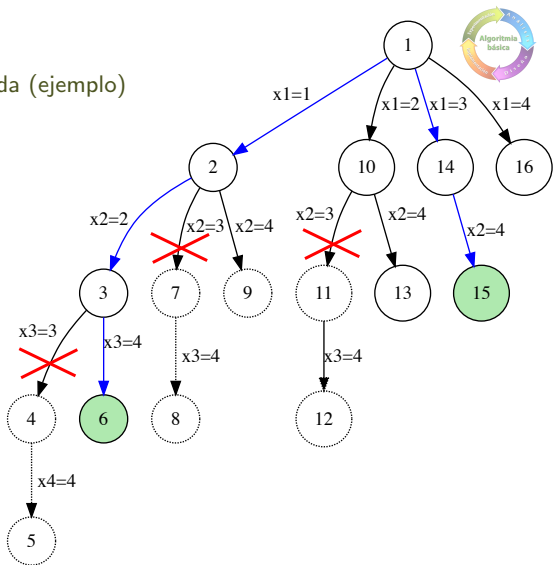
Problema: Dados n números positivos $a_i, 1 \leq i \leq n$, y un número positivo S encontrar todos los subconjuntos de los a_i , cuya suma es S .

- Una solución es un vector $(x_1, \dots, x_i), 1 \leq i \leq n$, donde cada $x_j, 1 \leq j \leq i$ es un índice que indica un elemento del conjunto
- **Restricciones explícitas:** $x_j \in \{1, \dots, n\}, 1 \leq j \leq i$
- **Restricciones implícitas**
 - 1 No hay repeticiones de índices y no hay repeticiones del mismo conjunto: $x_j < x_{j+1}, 1 \leq j \leq i-1$
 - 2 La suma de los elementos correspondientes es S : $\sum_{j=1}^i a_{x_j} = S$

Suma de subconjuntos

Representación del espacio de búsqueda (ejemplo)

• $x = (x_1, \dots, x_i)$

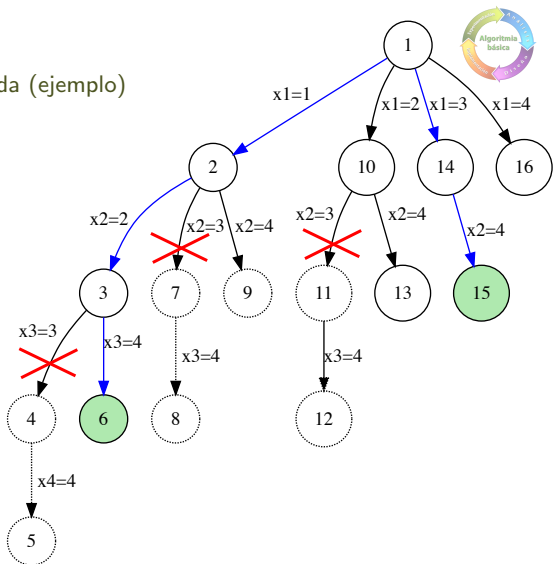


Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$

Suma de subconjuntos

Representación del espacio de búsqueda (ejemplo)

- $x = (x_1, \dots, x_i)$
- $G_i(x) = \{x_{i-1} + 1, \dots, 4\}$

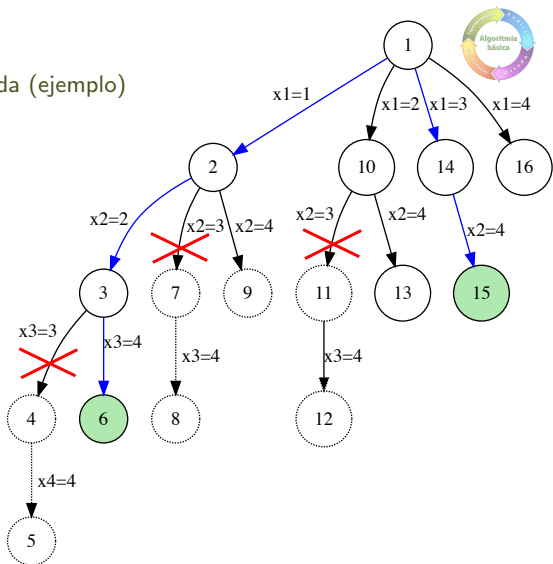


Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$

Suma de subconjuntos

Representación del espacio de búsqueda (ejemplo)

- $x = (x_1, \dots, x_i)$
- $G_i(x) = \{x_{i-1} + 1, \dots, 4\}$
- $P_i(x) : \sum_{j=1}^i a_{x_j} \leq S$

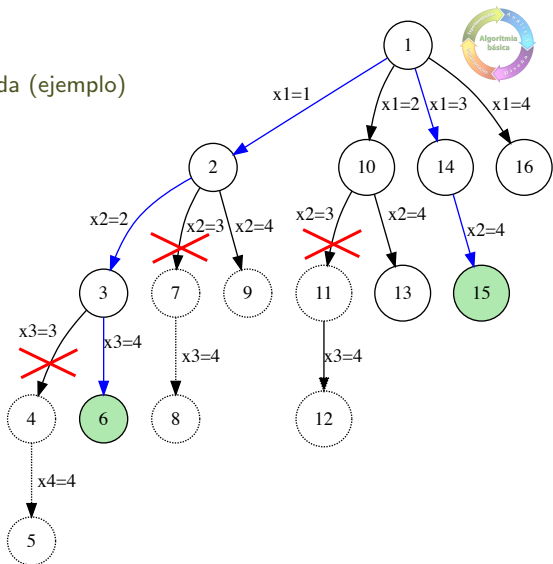


Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$

Suma de subconjuntos

Representación del espacio de búsqueda (ejemplo)

- $x = (x_1, \dots, x_i)$
- $G_i(x) = \{x_{i-1} + 1, \dots, 4\}$
- $P_i(x) : \sum_{j=1}^i a_{x_j} \leq S$
- $R_i(x) : \sum_{j=1}^i a_{x_j} = S$



Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$



Algoritmo

Encuentra todas las soluciones

```

1  // Pre: a es una tupla de n números positivos y  $S > 0$ ,
2  //        $1 \leq i \leq n$  y  $x[1, \dots, i]$  es una tupla de i índices
3  // Post: escribe todas las tuplas solución  $x[1, \dots, i]$ 
4
5  void sumaSubconjunto(tupla a, int S, tupla x, int i) {
6
7      for each nodo  $\in \{x[i-1]+1, \dots, n\}$  { //  $x[0] = 0$ 
8          suma_actual = suma(a, x, i-1, nodo); //  $\text{nodo} + \sum_{j=1}^{i-1} a_{x_j}$ 
9          if suma_actual  $\leq S$  { // predicado acotador
10             x[i] = nodo;
11             if suma_actual == S { escribir_solucion(x, i); }
12             if i < n { sumaSubconjunto(a, S, x, i+1); }
13         }
14     }
15 }
```

- La llamada inicial: `sumaSubconjunto(a, S, x, 1);`



Problema de la mochila 0-1

- Hay n objetos y una mochila
- El objeto i tiene peso $p_i > 0$ y su inclusión en la mochila produce un beneficio $b_i > 0$
- **Objetivo:** llenar la mochila de capacidad $C > 0$, de manera que se maximice el beneficio

$$\begin{aligned} \max \quad & \sum_{i=1}^n b_i x_i \\ \text{t.q.} \quad & \sum_{i=1}^n p_i x_i \leq C \\ & x_i \in \{0, 1\}, 1 \leq i \leq n \end{aligned}$$



Problema de la mochila 0-1

- Hay n objetos y una mochila
- El objeto i tiene peso $p_i > 0$ y su inclusión en la mochila produce un beneficio $b_i > 0$
- **Objetivo:** llenar la mochila de capacidad $C > 0$, de manera que se maximice el beneficio

$$\begin{aligned} \max \quad & \sum_{i=1}^n b_i x_i \\ \text{t.q.} \quad & \sum_{i=1}^n p_i x_i \leq C \\ & x_i \in \{0, 1\}, 1 \leq i \leq n \end{aligned}$$

Soluciones a problemas parecidos

- Objetos fraccionables ($0 \leq x_i \leq 1, 1 \leq i \leq n$): **solución voraz**
- Mochila 0-1 con pesos, beneficios y capacidad de la mochila números naturales (**solución de programación dinámica**)
- En el caso general, ninguna de las dos soluciones funciona



Representación de una solución

Hay dos formas:

- ① Tuplas de **tamaño variable** $(x_1, \dots, x_i), 1 \leq i \leq n$
 - Cada x_i es el índice del i -ésimo objeto introducido
 - El árbol de búsqueda será similar al ejemplo de la suma de subconjuntos



Representación de una solución

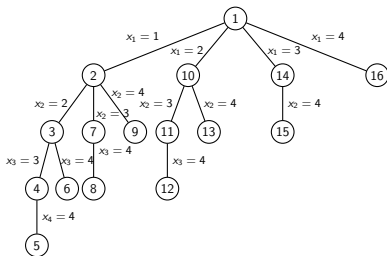
Hay dos formas:

- ① Tuplas de **tamaño variable** $(x_1, \dots, x_i), 1 \leq i \leq n$
 - Cada x_i es el índice del i -ésimo objeto introducido
 - El árbol de búsqueda será similar al ejemplo de la suma de subconjuntos
- ② Tuplas de **tamaño fijo** $(x_1, \dots, x_n)$
 - Cada $x_i \in \{0, 1\}$: $x_i = 1$ si el objeto i se introduce en la mochila, $x_i = 0$ otro caso
 - El árbol de búsqueda será un árbol binario

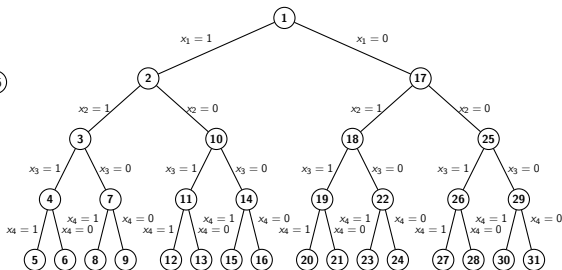


Representación del espacio de búsqueda

Ejemplo con $n = 4$ objetos



Tuplas de tamaño variable

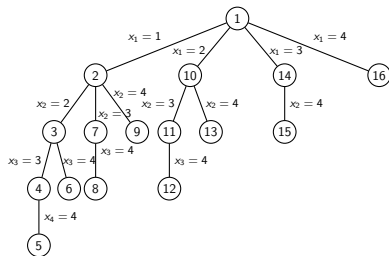


Tuplas de tamaño fijo

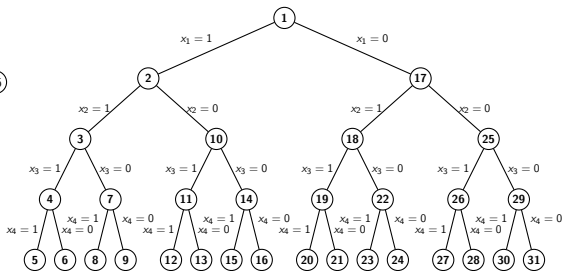


Representación del espacio de búsqueda

Ejemplo con $n = 4$ objetos



Tuplas de tamaño variable



Tuplas de tamaño fijo

En ambos casos, el recorrido en una búsqueda con retroceso será en profundidad



Restricciones

Representación de las soluciones como tuplas de tamaño fijo

- Una solución es un vector $(x_1, \dots, x_n)$, cada $x_i, 1 \leq i \leq n$, indica si se introduce o no el objeto i en la mochila
- **Restricciones explícitas:** $x_i \in \{0, 1\}, 1 \leq i \leq n$
- **Restricciones implícitas**
 - 1 La suma de los pesos de los objetos introducidos no puede superar la capacidad: $\sum_{i=1}^n p_i x_i \leq C$
 - 2 El beneficio total que se consigue con los objetos introducidos es máximo : $\sum_{i=1}^n b_i x_i = ??$



Predicados acotadores

Beneficio total máximo

Se quiere evitar generar ramas del árbol que no pueden producir soluciones mejores que la disponible actualmente

⇒ Se llama *poda basada en el beneficio de la mejor solución en curso*



Predicados acotadores

Beneficio total máximo

Se quiere evitar generar ramas del árbol que no pueden producir soluciones mejores que la disponible actualmente

⇒ Se llama *poda basada en el beneficio de la mejor solución en curso*

Poda

- 1 Calcular una cota superior del valor de la mejor solución posible al expandir un nodo y sus descendientes
- 2 Si la cota es menor o igual que el valor de la mejor solución disponible hasta el momento, no expandir ese nodo



Predicados acotadores

Cálculo de la cota superior

- Suponer que en el nodo actual se encuentre en el nivel $0 \leq i < n$
 - Ya se han determinado $x_j, 1 \leq j \leq i$
- Relajar el requisito de integridad del problema de la mochila 0-1:
 - Para todo x_j por determinar ($i+1 \leq j \leq n$), x_j puede tomar valores reales en $[0, 1]$
- Aplicar el algoritmo voraz . . .



Algoritmo voraz para el cálculo de la cota superior

Repaso I

- Definimos el tipo de dato **Mochila**: $\{b, p, C, n\}$
 - b (beneficios) es una tupla de n reales (positivos)
 - p (pesos) es una tupla de n reales (positivos)
 - C (capacidad) es un número real (positivo)
 - n es el número de objetos
- Los objetos de una *mochila* están ordenados según el beneficio unitario:

$$\forall i = 1, \dots, mochila.n-1 : \frac{mochila.b[i]}{mochila.p[i]} \geq \frac{mochila.b[i+1]}{mochila.p[i+1]}$$



Algoritmo voraz para el cálculo de la cota superior

Repaso II

```
1  // Pre: en mochila, los objetos están ordenados según el beneficio unitario
2  //      libre es la capacidad aún libre de la mochila,
3  //      ben es el beneficio actual,
4  //      i es el índice del primer objeto a considerar
5
6  double cota(Mochila mochila, double libre, double ben, int i) {
7      if i>mochila.n or libre=0.0 { return ben; }
8      else {
9          if mochila.p[i] > libre
10             { return ben + libre/mochila.p[i]*mochila.b[i]; }
11         else
12             { return cota(mochila, libre-mochila.p[i], ben+mochila.b[i], i+1); }
13     }
14 }
```



Algoritmo de búsqueda

- Definimos el tipo de dato **Sol**: $\{x, ben, peso\}$
 - x es una tupla de enteros $\{0, 1\}$
 - ben es el beneficio total asociado a la tupla
 - $peso$ es el peso total asociado a la tupla



Algoritmo de búsqueda

```

1  // Pre: en mochila, los objetos están ordenados según el beneficio unitario
2  //      actual es una solución prometedora
3  //      mejor es la mejor solución hasta ahora
4  //      i es el índice del objeto
5  void buscaSolución(Mochila mochila, Sol actual, Sol mejor, int i) {
6    for each decisión ∈ {1,0} {
7      totPeso = actual.peso + mochila.p[i] * decisión
8      totBen = actual.ben + mochila.b[i] * decisión
9      if totPeso <= mochila.C and // P1 and
10         mejor.ben < cota(mochila,mochila.C-totPeso,totBen,i+1) { // P2
11         actual.x[i] = decisión;
12         actual.peso = totPeso;
13         actual.ben = totBen;
14         if i == mochila.n // es hoja
15             if actual.ben > mejor.ben {copia(mejor,actual);}
16         else { buscaSolución(mochila, actual, mejor, i+1); }
17     }
18 }
19 }
```



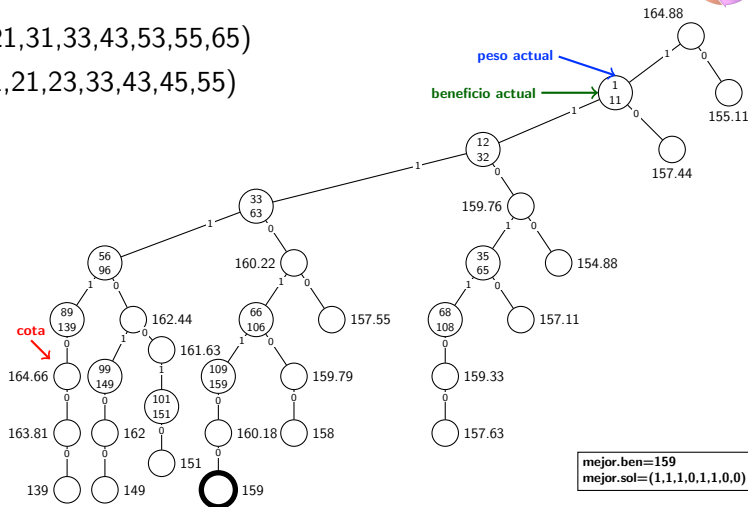
Algoritmo de búsqueda

Llamada inicial

```
1 // Pre: en mochila, los objetos están ordenados según el beneficio unitario
2
3 void resolverMochila0-1(Mochila mochila) {
4
5     inicializa(actual); // solución (parcial) actual
6     inicializa(mejor); // mejor solución
7     buscaSolución(mochila, actual, mejor, 1);
8
9 }
```

Ejemplo

- $b=(11,21,31,33,43,53,55,65)$
- $p=(1,11,21,23,33,43,45,55)$
- $C=110$
- $n=8$





Problema de la mochila 0-1

Número de nodos generados en el ejemplo:

Fuerza bruta

510 nodos



Problema de la mochila 0-1

Número de nodos generados en el ejemplo:

Fuerza bruta	510 nodos
Backtracking ($P1$)	328 nodos ☹



Problema de la mochila 0-1

Número de nodos generados en el ejemplo:

Fuerza bruta	510 nodos
Backtracking ($P1$)	328 nodos ☹
Backtracking ($P1$ y $P2$)	26 nodos ☺



Problema de la mochila 0-1

Número de nodos generados en el ejemplo:

Fuerza bruta	510 nodos
Backtracking ($P1$)	328 nodos ☹
Backtracking ($P1$ y $P2$)	26 nodos ☺

Para poder aplicar $P2$ (cota calculada con solución voraz) los objetos tienen que estar ordenados según el beneficio unitario

⇒ de mayor a menor



Técnica *meet-in-the-middle*

- Los predicados acotadores en la búsqueda con retroceso juegan un papel importante en la eficiencia del algoritmo
 - En particular, en el tiempo de ejecución



Técnica *meet-in-the-middle*

- Los predicados acotadores en la búsqueda con retroceso juegan un papel importante en la eficiencia del algoritmo
 - En particular, en el tiempo de ejecución
 - Sin embargo, no siempre son suficientes para conseguir tiempos de solución razonables
- ⇒ La técnica *meet-in-the-middle* puede mejorar la eficiencia



Técnica *meet-in-the-middle*

Enfoque solución

- ❶ Se divide el espacio de búsqueda en dos partes de igual tamaño aprox.
 - ❷ Se realiza la búsqueda en las dos partes por separado y
 - ❸ Se combinan los resultados de las dos búsquedas
- La búsqueda en cada parte puede ser por fuerza bruta, búsqueda con retroceso, etc.
 - Se utiliza cuando existe una forma eficiente de combinar los resultados



Técnica *meet-in-the-middle*

Ejemplo de aplicación combinado con fuerza bruta

(<https://www.geeksforgeeks.org/meet-in-the-middle/>)

Suma de subconjuntos

Dados n números positivos $a_i, 1 \leq i \leq n$, y un número positivo S encontrar todos los subconjuntos de los a_i , cuya suma es S .

⇒ Tiempo de búsqueda $O(2^n)$



Técnica *meet-in-the-middle*

Ejemplo de aplicación combinado con fuerza bruta

(<https://www.geeksforgeeks.org/meet-in-the-middle/>)

Suma de subconjuntos

Dados n números positivos $a_i, 1 \leq i \leq n$, y un número positivo S encontrar todos los subconjuntos de los a_i , cuya suma es S .

⇒ Tiempo de búsqueda $O(2^n)$

Enfoque solución

- 1 Dividir la lista de número en dos listas A y B de igual tamaño aprox
- 2 Generar los subconjuntos de A y guardar sus sumas en la lista S_A
- 3 Generar los subconjuntos de B y guardar sus sumas en la lista S_B
- 4 Ordenar las dos listas S_A y S_B
- 5 Encontrar las parejas $(a, b), a \in S_A$ y $b \in S_B$ tal que $a + b = S$



Técnica *meet-in-the-middle*

Ejemplo de aplicación combinado con fuerza bruta $\Rightarrow O(n \cdot 2^{\frac{n}{2}})$

Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$



Técnica *meet-in-the-middle*

Ejemplo de aplicación combinado con fuerza bruta $\Rightarrow O(n \cdot 2^{\frac{n}{2}})$

Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$

- ➊ Dividir la lista ($\Rightarrow O(1)$)
 - $A = \{11, 13\}$, $B = \{24, 7\}$



Técnica *meet-in-the-middle*

Ejemplo de aplicación combinado con fuerza bruta $\Rightarrow O(n \cdot 2^{\frac{n}{2}})$

Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$

- ➊ Dividir la lista ($\Rightarrow O(1)$)
 - $A = \{11, 13\}$, $B = \{24, 7\}$
- ➋ Generar subconjuntos de A y la lista S_A ($\Rightarrow O(2^{\frac{n}{2}})$)
 - $S_A = \{\{\} : 0, \{11\} : 11, \{13\} : 13, \{11, 13\} : 24\}$



Técnica *meet-in-the-middle*

Ejemplo de aplicación combinado con fuerza bruta $\Rightarrow O(n \cdot 2^{\frac{n}{2}})$

Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$

- ➊ Dividir la lista ($\Rightarrow O(1)$)
 - $A = \{11, 13\}$, $B = \{24, 7\}$
- ➋ Generar subconjuntos de A y la lista S_A ($\Rightarrow O(2^{\frac{n}{2}})$)
 - $S_A = \{\{\} : 0, \{11\} : 11, \{13\} : 13, \{11, 13\} : 24\}$
- ➌ Generar subconjuntos de B y la lista S_B ($\Rightarrow O(2^{\frac{n}{2}})$)
 - $S_B = \{\{\} : 0, \{24\} : 24, \{7\} : 7, \{24, 7\} : 31\}$



Técnica *meet-in-the-middle*

Ejemplo de aplicación combinado con fuerza bruta $\Rightarrow O(n \cdot 2^{\frac{n}{2}})$

Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$

- ➊ Dividir la lista ($\Rightarrow O(1)$)
 - $A = \{11, 13\}$, $B = \{24, 7\}$
- ➋ Generar subconjuntos de A y la lista S_A ($\Rightarrow O(2^{\frac{n}{2}})$)
 - $S_A = \{\{\} : 0, \{11\} : 11, \{13\} : 13, \{11, 13\} : 24\}$
- ➌ Generar subconjuntos de B y la lista S_B ($\Rightarrow O(2^{\frac{n}{2}})$)
 - $S_B = \{\{\} : 0, \{24\} : 24, \{7\} : 7, \{24, 7\} : 31\}$
- ➍ Ordenar las listas ($\Rightarrow O(2^{\frac{n}{2}} \cdot n)$) = ($\Rightarrow O(n \cdot 2^{\frac{n}{2}})$)
 - $S_A = \{\{\} : 0, \{11\} : 11, \{13\} : 13, \{11, 13\} : 24\}$
 - $S_B = \{\{\} : 0, \{7\} : 7, \{24\} : 24, \{24, 7\} : 31\}$



Técnica *meet-in-the-middle*

Ejemplo de aplicación combinado con fuerza bruta $\Rightarrow O(n \cdot 2^{\frac{n}{2}})$

Ejemplo: $n = 4$, $\{11, 13, 24, 7\}$ y $S = 31$

- ➊ Dividir la lista ($\Rightarrow O(1)$)
 - $A = \{11, 13\}$, $B = \{24, 7\}$
- ➋ Generar subconjuntos de A y la lista S_A ($\Rightarrow O(2^{\frac{n}{2}})$)
 - $S_A = \{\{\} : 0, \{11\} : 11, \{13\} : 13, \{11, 13\} : 24\}$
- ➌ Generar subconjuntos de B y la lista S_B ($\Rightarrow O(2^{\frac{n}{2}})$)
 - $S_B = \{\{\} : 0, \{24\} : 24, \{7\} : 7, \{24, 7\} : 31\}$
- ➍ Ordenar las listas ($\Rightarrow O(2^{\frac{n}{2}} \cdot n)$) = ($\Rightarrow O(n \cdot 2^{\frac{n}{2}})$)
 - $S_A = \{\{\} : 0, \{11\} : 11, \{13\} : 13, \{11, 13\} : 24\}$
 - $S_B = \{\{\} : 0, \{7\} : 7, \{24\} : 24, \{24, 7\} : 31\}$
- ➎ Encontrar las parejas ($\Rightarrow O(n \cdot 2^{\frac{n}{2}})$)
 - $\{\{\} : 0, \{24, 7\} : 31\} \in S_A \times S_B \Rightarrow \{24, 7\}$
 - $\{\{11, 13\} : 24, \{7\} : 7\} \in S_A \times S_B \Rightarrow \{11, 13, 7\}$